Deterministic per-relink PPS on the CTS/KR260 GTHE4: LPDC revived, the real root causes, and the ±50 ps result

Date: 2026-08-19 **Status:** Working, committed (`wr-cores ba5173d4`, `wrpc-sw 25a79ba9 + 27bdf6c8`) **Target:** CTS / KR260, Xilinx GTHE4 (Kria K26), White Rabbit

Scope. This note records the second, successful LPDC-on-RXPI campaign. The first attempt (lpdc-rxpi-phase-matching.md, 2026-08-14) achieved a deterministic link layer but concluded that the remaining 2–8 ns per-relink phase scatter was inherent to the RXPI direct-tag (a phase reference tied to the random CDR lock) and could only be fixed with a DMTD clock; the work was reverted. **That conclusion was wrong.** The scatter had three concrete, fixable causes — none of them the direct-tag: an uncalibrated master-side timestamp phase (t4), an uncompensated per-relink RX bit-slide latency, and a GTHE4-specific wrap in that latency. With all three fixed, the per-relink PPS offset between two CTS nodes is **constant to ±50 ps across relinks and power cycles**, with no DMTD clock. This note documents the diagnosis chain — each cause was isolated from on-hardware relink datasets before it was fixed — and the final design.

---

1. Why the 2026-08-14 verdict was re-examined

The reversal blamed the direct-tag: “nothing pins the sub-UI recovered-clock phase reproducibly.” That verdict predates the TXPI bench work (2026-08-17), which built an MMCM vernier phase meter (`top/cts/txpi.c`) and showed the **recovered clock phase is constant per relink (±28 ps)** while the PPS offset scattered on a coarse ~1.6 ns grid. If the direct-tag reference were truly random per relink, the meter would scatter too; it does not. The coarse term therefore had to live in the *datapath or the measurement*, not in the clock — which reopened the deterministic-latency (LPDC) direction, this time with the meter available to tell the difference.

2. What was built (differences from the first attempt)

The RX datapath is the same idea as the first attempt — elastic buffer bypassed, RAW 20-bit, fabric comma alignment — rebuilt self-contained so the pure-RXPI firmware architecture is kept:

LAYER	DESIGN
GT IP (`gthe4_phy.tcl`)	`RX_BUFFER_MODE 0`, `RX_OUTCLK_SOURCE RXOUTCLKPMA`, `RX_SLIDE_MODE PCS`, `RX_DATA_DECODING RAW` / 20-bit, buffbypass controller in `CORE`; QPLL0 fractional-N, `dmonitor`, `txpippm` preserved. TX stays GT-internal 8b10b, buffered
Fabric (`rxpi_lp_adapter.vhd`, new)	raw 20-bit → `gtx_comma_detect_lp` (fixed target tap **0**) → 2× `gc_dec_8b10b`; a fabric FSM steers the comma to tap 0 with PCS `rxslide` pulses (**2 RXUSRCLK2 cycles high** each — UG578 minimum; 1-cycle pulses can be ignored). `serdes_ready` is gated on `rxcdrlock` so unplug drops `PHY_READY`; buffbypass + comma-detector reset is held until `rxresetdone`
Register access	none — comma target is a generic (0), status/diagnostics through the existing AUX-WB `rxpi_gthe4_map` (`bitslide[31:8]`: tap, aligned, windowed 8b10b error count, last failing symbol)
Firmware (`lpdc_gthe4_rxpi.c`)	pure-RXPI driver kept; AN restarted on comma accept (wall #5 of the first note); `RX_WAIT_COMMA` timeout re-throw as a safety net

Tap 0 is the only clean framing. `gtx_comma_detect_lp`'s barrel-shift output window `merged(t+19:t)` stitches *previous*-word bits where *next*-word bits belong for any target t>0. Periodic idles mask this, but non-periodic data — the `/C1//C2/` auto-negotiation marker — is corrupted (wall #6 of the first note was one symptom of this). So the comma must be *physically steered* to tap 0 by `rxslide`; the barrel-shift is then a pass-through.

A diagnostics lesson. One full detour this round (a fabric-RAW TX conversion, later reverted) was caused by a debug error counter that never cleared and counted during pre-alignment — where misframing is definitionally expected — so it read saturated even on a healthy link. The counter is now windowed (~0.13 s) and gated on `aligned`. Instrument the *steady state*, not the history since reset.

3. Root cause 1 — the master never calibrates its RX timestamp phase

With the link deterministic (tap 0, clean decode, replug-safe), the PPS still scattered over ~9 ns. Six relinks, slave `stat` alongside the scope:

PPS (NS)	DMS = CRTT/2 (NS)	PPS + DMS (NS)
3.29	341.79	345.1
3.64	339.00	342.6
1.50	339.60	341.1
10.29	332.40	342.7
4.68	339.60	344.3
4.79	334.70	339.5

PPS anti-correlates ~1:1 with dms: the raw spread of 8.8 ns collapses to ±2.8 ns in PPS+dms, and crtt itself moved **19 ns on a fixed fiber**. The scatter was in the *measured link delay*; PTP faithfully steered the slave to the wrong place by −Δcrtt/2.

The mechanism, in code: every RX timestamp — the slave's t2 **and the master's t4** — is de-quantized with the `ptrackers[0]` phase (`lib/net.c`). In direct-tag mode that phase has an arbitrary zero per relink; the RXPI sweep exists to calibrate `ptrackers[0].offset`. But the sweep FSM gated on `SEQ_WAIT_MAIN` → `mpll.phase_ld.locked`, and a free-running master goes straight to `SEQ_READY` with the mpll disabled — **the master never ran the sweep** (its `ps_ctrl`/`ps_res` registers were still at reset values). Uncalibrated t4 phase → crtt error up to one 16 ns clock period per relink → PPS error up to ±8 ns.

Fix (firmware): a new `RX_WAIT_SYNTON` state for the master role. The sweep needs a syntonized RX clock (the remote slave locked to us), which the master detects as a *stationary ptracker phase* (<200 ps drift per 500 ms, three consecutive, wrap-aware at 16 000 ps). It then runs the same sweep but applies **only** `ptrackers[0].offset` — the mpll is never touched; the master's oscillator stays the reference. (`mpll.tag_ref`, the sweep's fine-phase stitch input, updates on masters too: `mpll_update` stores it before its enabled/link-up bail-out.)

4. Root cause 2 — each rxslide is 1 UI of uncompensated RX latency

With the master calibrated, the next dataset got *clean but quantized*: PPS+dms landed on an **exact 800 ps grid** (pairs identical to 10 ps), and crtt offsets were also exact UI multiples. Measurement was now ps-accurate; the remaining variable was physical: **each PCS `rxslide` shifts RX latency by one UI**, and the slide count needed to reach tap 0 is random per CDR lock (wherever the comma lands), independently on both ends.

This is precisely the term classic White Rabbit compensates with `gtp_bitslide` — and the CTS board tied `rx_bitslide` to zero.

Fix (gateware + firmware): the adapter counts the slides it issues and exports the count as `rx_bitslide`; the board feeds it to the endpoint. The consumer path already existed end-to-end: endpoint MDIO

`WR_SPEC.BSLIDE` → `ep_get_bitslide()` (×800 ps) → PPSi `semistaticLatency`, applied by each end to its **own** RX timestamps (slave t2, master t4), and re-read on every link-up (`wrc_check_link` → `wrc_ptp_start`). The slide count is final before auto-negotiation completes, so the value read is always current. The firmware's GTX-era "reject odd bitslide" re-throw was removed — with real slide counts it would reject odd values forever.

5. Root cause 3 — the GTHE4 PCS slide latency wraps at the 10-bit symbol

Eight relinks with compensation active: five sat at PPS ≈ 4.98 ns, three at ≈ 8.94 ns — and the split correlated *exactly* with the master's slide count:

MASTER SLIDE COUNT	CRTT (PS)	PPS (NS)
`0–9 (five relinks)`	665 79x ±100 ps	4.97 ±0.05
`11, 18, 18`	657 79x — exactly 10 UI (8.0 ns) low	8.9x

Announcing a raw count of 11 or 18 over-corrects crtt by exactly 10 UI: **the GTHE4 PCS slide latency wraps at the 10-bit symbol, not the 20-bit word** — the physical added latency is (n mod 10) UI. Both outlier values fit exactly (18→8, 11→1). This is a hardware-measured property; it is not stated in UG578.

Fix (gateware, one line): the exported slide count wraps at 10.

6. Result

Eight relinks after the mod-10 fix (slide counts 0–9 on both ends, every combination including both ends at 9):

#	PPS (NS)	MASTER/SLAVE SLIDES	CRTT (PS)
1	4.99	0 / 8	665 790
2	4.98	7 / 1	665 794
3	4.89	9 / 5	665 590
4	4.99	8 / 2	665 793
5	4.98	0 / 7	665 794
6	4.97	3 / 0	665 792
7	4.99	9 / 9	665 792
8	4.99	8 / 9	665 793

- **PPS offset constant at 4.98 ns ±50 ps across relinks, and across power cycles.** (Down from ~8 ns of scatter at the start of the campaign.)
- crtt in a single cluster, spread ±102 ps. Row 3 is exactly one 200 ps sweep-stitch grain low in crtt and grain/2 in PPS — the system is coherent down to the stitch quantum.
- Frequency syntonization unchanged throughout (~9 ps rms, as always).

7. What remains

ITEM	NATURE
The 4.98 ns constant	Static calibration, stable across power cycles — absorb into the SFP deltas (`sfp add`) / t24p like any WR deployment; PPS then lands at ~0
±50 ps residual	The sweep coarse/fine stitch quantum (200 ps). Optional: harden the stitch heuristic, or use the TXPI park (`txpi.c rpark` , demonstrated ±14 ps on the bench) for the last tens of ps
TXPI servo	The TXPIPPM actuator is wired in gateware and driven from the PS bench tool only; no firmware servo — deliberate for now

8. Correction to the 2026-08-14 note

lpdc-rxpi-phase-matching.md §6 concluded the RXPI direct-tag references the random CDR lock phase and that sub-ns reproducibility required a DMTD clock; §7 reverted the work on that basis. Both are **disproven by this campaign**: the direct-tag path, once the master sweep calibrates t4 and the rxslide UI latency is announced (mod 10), delivers per-relink PPS deterministic to ±50 ps with no DMTD. The first note's *observations* (the seven walls, the `fixoff` experiment showing offset scatter) were all correct — the `fixoff` scatter is now explained by the then-uncalibrated master t4 and unannounced bitslide, which no slave-side pinning could fix. That note remains a valid record of the link-layer bring-up; only its root-cause conclusion and the "reverted, needs DMTD" status are superseded.